# PRE-LAB 06 — From data points to paths

**15–20 minutes before class · D3 v7 · Two runnable examples**

**Goal:** Understand why a line chart has **one SVG path per series**, how time is encoded, and why a stacked area displays a **sum**. This pre-lab uses **fictional data**, not COVID data. The full COVID task belongs to Lab 06.

## Run first (2 minutes)

1. Open `examples/01_worked_multiline.html` in a browser with Internet access (D3 CDN). If necessary, start `python -m http.server 8000` in this folder and open the file via `http://localhost:8000/examples/01_worked_multiline.html`.
2. Open **F12 → Console**; confirm the cleaned records and two grouped series are printed.
3. Move the pointer across the chart and hover the **North** and **South** legend items. What changes?

## Example A: two time series (10 minutes)

**Mental model:**

```
10 rows (city, date, value)
     ↓ d3.group(..., city)
2 groups, each with 5 sorted dates
     ↓ .data(series) + d3.line()(group.values)
2 SVG <path> elements
     ↓ pointer → date → nearest observation
read BOTH city values at one date
```

Look for the numbered `STEP 1–6` comments in `01_worked_multiline.html`:

- **STEP 1:** `d3.timeParse('%Y-%m-%d')` converts date strings to `Date` objects. `console.table()` helps verify values.
- **STEP 2:** `d3.group(tidy, d => d.city)` creates a map of series. **Sort dates** before drawing the path.
- **STEP 3:** `d3.scaleTime()` maps dates to horizontal pixels; `d3.scaleLinear()` maps counts to vertical pixels. Larger values appear **higher** because Y is inverted.
- **STEP 4:** `d3.line().x(...).y(...)` creates the path geometry; `.data(series)` joins one `<path>` to each group. The path contains many points.
- **STEP 5:** Legend hover reduces opacity of nonselected series; it does not remove their data.
- **STEP 6:** `x.invert(pixelX)` estimates the date; a bisector selects the closest record. The readout shows values **across all lines** at that date.

**Try it:** Change North's value on `2020-04-03` from `24` to `38`, reload, and explain which line, y-axis maximum and readout should change. Next, comment out `.sort((a,b)=>a.date-b.date)` and predict the result if input rows are shuffled.

## Example B: one stacked area (5 minutes)

Open `examples/02_worked_stacked_area.html`. Click **Hide / show Workshops**.

```
Rows: date + Lectures + Workshops
          ↓ d3.stack().keys([...])
Each category becomes a [bottom, top] layer
          ↓ d3.area().y0(...).y1(...)
Two colored areas; the TOP boundary is the SUM
```

At 1 April: `Lectures = 18`, `Workshops = 8`, therefore the **top of the stack = 26**. The orange band's **thickness** represents 8, not its absolute Y position. `d3.stack()` requires aligned categories at each date and an appropriate input shape.

## Before class: answer briefly (2–3 minutes)

1. Why does a five-day, two-city chart have **two paths**, not ten?
2. Why should you **not** give a date string directly to `scaleTime()` without parsing it?
3. If the cursor points at 3 April, how does the shared readout retrieve values from both series?
4. In a stacked area, which visual measurement shows a category value: the **top height** or the **band thickness**?

**Connection to Lab 06:** Your real CSV has *one column per date* and may have *multiple rows per country*. You must first transform it into `Country, Date, confirmed cases`, aggregate the four countries correctly and verify the values. Do **not** copy this fictional dataset as the lab answer.